

This is a copy of a shared ChatGPT chat. It won't be added to memory.

Report conversation

Ya, alur eksperimen Anda **sudah benar** dan merupakan desain yang kuat untuk evaluasi patient-independent.

Strukturnya sebaiknya seperti ini:

1. Split awal berbasis pasien: 80% development/training + 20% unseen test.

Semua record dari satu pasien harus masuk hanya ke salah satu bagian. Tidak boleh ada record pasien yang sama muncul di 80% dan 20%.

2. Pada 80% development set, lakukan 5-fold cross-validation berbasis pasien.

Jadi setiap fold kira-kira menggunakan 64% total pasien untuk training dan 16% total pasien untuk validation. Dari sini Anda memperoleh distribusi performa, misalnya:

$$\text{Macro-F1} = 0.xx \pm 0.xx$$

dan memilih hyperparameter/model configuration.

3. Setelah konfigurasi terbaik ditentukan, jangan mengambil salah satu model dari 5 fold sebagai model final. Train ulang model dari awal menggunakan **seluruh 80% development patients** dengan konfigurasi yang sudah dipilih.

4. Model final tersebut kemudian digunakan sekali untuk inferensi pada 20% unseen patients:

$$80\% \text{ Development} \rightarrow \begin{cases} 5\text{-fold CV} \\ 100\% \text{ of development} \end{cases} \begin{matrix} \text{model selection} \\ \text{final training} \end{matrix} \rightarrow 20\% \text{ u}$$

Ada satu hal yang sangat penting untuk riset cardiac localization Anda: **semua preprocessing yang belajar dari data harus berada di dalam fold CV**, misalnya normalization/scaler, feature selection, PCA, SMOTE/oversampling, data-driven threshold selection, dan sejenisnya. Jangan menghitung parameter preprocessing menggunakan seluruh 80% lalu baru menjalankan KCV, karena itu dapat menyebabkan leakage antar-fold.

Untuk kasus beberapa record berasal dari pasien yang sama, pembagiannya idealnya seperti:

```

140 patients
|
├─ Development: 112 patients (80%)
|   |
|   ├─ Fold 1: train patients | validation patients
|   ├─ Fold 2: train patients | validation patients
|   ├─ Fold 3: train patients | validation patients
|   ├─ Fold 4: train patients | validation patients
|   └─ Fold 5: train patients | validation patients
|
|   Hyperparameter/configuration selection
|
|   Final model ← train on ALL 112 patients
|
└─ Unseen Test: 28 patients (20%)
    ↓
    Final inference
    ↓
    Accuracy / Macro-F1 / Sensitivity / Specificity /
    AUROC / confusion matrix / localization metrics
  
```

20% test itu sebaiknya benar-benar “dikunci”. Jangan digunakan untuk menentukan epoch, threshold, architecture, hyperparameter, feature selection, ataupun memilih model mana yang akan dilaporkan. Kalau setelah melihat test Anda mengubah model lalu mengetes lagi berkali-kali, test tersebut secara praktis berubah menjadi validation set.

Jadi, secara metodologis desain Anda adalah **patient-level holdout + patient-level 5-fold CV pada development set + final retraining + independent holdout evaluation**. Ini jauh lebih kuat daripada hanya melakukan random record-level 5-fold CV, terutama untuk sinyal cardiac karena beberapa record dari pasien yang sama biasanya sangat berkorelasi.

Jika saya ingin tuning modelnya sehingga dapat kinerja terbaik, pada model yang mana, atau proses yang mana saya perlu lakukan? Coba jelaskan saya kurang faham.

Tuning dilakukan **di dalam 80% development data**, tepatnya menggunakan hasil **5-fold cross-validation**. Jangan tuning menggunakan model final yang sudah dilatih dengan seluruh 80%, dan jangan pernah menggunakan 20% unseen test untuk mencari konfigurasi terbaik.

Bayangkan alurnya begini:

1. Anda punya 100% pasien.
 - 80% = development/training pool
 - 20% = unseen test, dikunci sampai akhir
2. Pada 80% development pool, Anda mencoba berbagai konfigurasi model. Misalnya:
 - learning rate: 0.001, 0.0005, 0.0001
 - batch size: 16, 32, 64
 - jumlah layer
 - jumlah filter
 - dropout
 - optimizer
 - attention dimension
 - loss function
 - dan sebagainya.
3. Untuk **setiap kombinasi hyperparameter**, jalankan 5-fold CV berbasis pasien.

Contohnya:

Configuration A

LR = 0.001

Dropout = 0.3

Batch = 32

Fold-1 → Macro-F1 = 0.81

Fold-2 → Macro-F1 = 0.84

Fold-3 → Macro-F1 = 0.79

Fold-4 → Macro-F1 = 0.83

Fold-5 → Macro-F1 = 0.82

Mean Macro-F1 = 0.818

Kemudian:

Configuration B

LR = 0.0005

Dropout = 0.4

Batch = 32

Fold-1 → 0.84

Fold-2 → 0.86

Fold-3 → 0.83

Fold-4 → 0.85

Fold-5 → 0.84

Mean Macro-F1 = 0.844

Lalu Configuration C, D, dan seterusnya.

Misalnya hasil akhirnya:

Configuration	LR	Dropout	Batch	Mean CV Macro-F1
A	0.001	0.3	32	0.818
B	0.0005	0.4	32	0.844
C	0.0001	0.3	16	0.826
D	0.0005	0.5	64	0.831

Maka **Configuration B** dipilih.

Baru setelah itu Anda membuat **model final baru** dari awal:

```
Best configuration dari CV
    ↓
LR = 0.0005
Dropout = 0.4
Batch = 32
    ↓
Train ulang menggunakan SELURUH 80% pasien
    ↓
Final Model
    ↓
Inferensi sekali pada 20% unseen patients
```

Jadi ada perbedaan penting antara **model-model untuk tuning** dan **model final**.

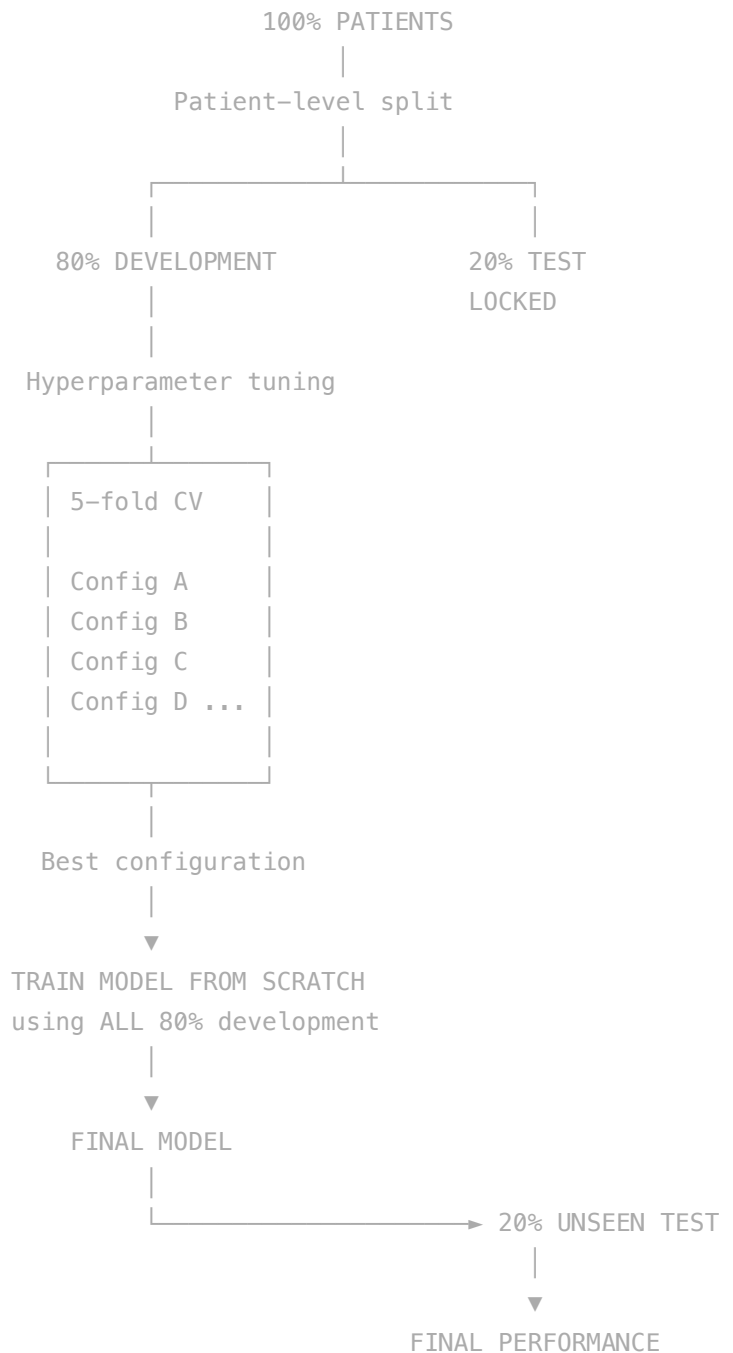
Model yang muncul pada Fold 1–5 sebenarnya hanya "model eksperimen". Mereka digunakan untuk menjawab pertanyaan:

"Konfigurasi model mana yang kemungkinan memiliki generalisasi terbaik?"

Setelah pertanyaan itu terjawab, kelima model tersebut pada prinsipnya tidak menjadi model akhir.

Model final dibuat **setelah tuning selesai**, menggunakan parameter terbaik tadi dan seluruh 80% development data.

Skema lengkapnya kira-kira:



Hal yang sering membuat bingung adalah **epoch**. Misalnya hasil CV menunjukkan konfigurasi terbaik menggunakan LR 0.0005, dropout 0.4, batch 32. Tetapi Anda juga perlu menentukan berapa epoch model final dilatih.

Ada beberapa cara. Salah satu yang sederhana dan cukup defensible adalah selama CV gunakan **early stopping**. Misalnya best epoch dari lima fold:

Fold 1 → epoch 43
Fold 2 → epoch 51
Fold 3 → epoch 47
Fold 4 → epoch 45
Fold 5 → epoch 49

Median $\approx$ 47 epoch

Kemudian model final dapat dilatih pada seluruh 80% selama sekitar **47 epoch** menggunakan konfigurasi terpilih. Dengan begitu Anda tidak perlu "mengintip" 20% test untuk menentukan epoch.

Yang paling penting, jangan melakukan pola seperti:

```
Train 80%
↓
test pada 20%
↓
Macro-F1 kurang bagus
↓
ubah dropout
↓
test lagi pada 20%
↓
ubah learning rate
↓
test lagi
```

Ini **salah secara metodologis**, karena Anda secara tidak langsung melakukan tuning terhadap test set. Setelah beberapa kali seperti itu, 20% tadi bukan lagi benar-benar unseen.

Untuk riset lokalisasi cardiac Anda, saya sarankan secara konseptual membagi proses menjadi tiga tahap:

Tahap 1 — Model development. Semua eksperimen arsitektur, ablation, optimizer, learning rate, dropout, loss, threshold, dan sebagainya dilakukan hanya di 80% dengan patient-level 5-fold CV.

Tahap 2 — Model selection. Pilih satu konfigurasi berdasarkan metrik utama yang sudah ditentukan sebelumnya, misalnya Macro-F1 jika kelas lokasi cardiac tidak seimbang.

Tahap 3 — Final evaluation. Rebuild model dengan konfigurasi terpilih pada seluruh 80%, kemudian sekali evaluasi terhadap 20% holdout.

Jadi jawabannya singkatnya:

— **Tuning bukan dilakukan pada final model. Tuning dilakukan melalui 5-fold CV pada 80% development set. Hasil tuning menentukan konfigurasi n**  **Upgrade** 
Setelah itu model final dilatih ulang menggunakan seluruh 80%, lalu diuji sekali pada 20% unseen test.

Kalau model cardiac localization Anda nanti punya banyak hyperparameter, saya juga bisa bantu menentukan **parameter mana yang benar-benar perlu dituning dan mana yang sebaiknya dibuat fixed**, supaya eksperimennya tidak meledak menjadi ratusan kombinasi.

Ok faham, kalau saya pakai grid search berarti akan semakin banyak parameter yang saya coba ya?

Betul. Dengan **grid search**, Anda mendefinisikan beberapa nilai untuk setiap hyperparameter, lalu sistem mencoba **semua kombinasi** yang mungkin.

Misalnya Anda punya:

- Learning rate: 3 nilai
- Batch size: 3 nilai
- Dropout: 3 nilai
- Jumlah filter: 2 nilai

Maka jumlah kombinasi:

$$3 \times 3 \times 3 \times 2 = 54$$

Karena setiap kombinasi dievaluasi dengan **5-fold CV**, total training menjadi:

$$54 \times 5 = 270 \text{ training runs}$$

Jadi benar, jumlah eksperimen bisa cepat sekali membesar.

Untuk riset cardiac localization, biasanya lebih efisien kalau grid-nya tidak terlalu lebar. Misalnya fokus dulu pada 3–5 hyperparameter yang paling berpengaruh, seperti learning rate, dropout, batch size, jumlah filter/channel, dan mungkin weight decay. Parameter arsitektur lain yang tidak terlalu kritis bisa dibuat fixed berdasarkan desain awal.

Kalau search space sudah besar, **random search** atau **Bayesian optimization/Optuna** sering lebih hemat komputasi daripada exhaustive grid search. Namun untuk paper, grid search kecil dan terkontrol justru enak karena sederhana, transparan, dan mudah direproduksi.

Kalau Anda mau, saya bisa bantu buat **grid search yang realistis untuk model cardiac localization Anda**, misalnya target hanya sekitar **30–60 kombinasi**, bukan ratusan.

If you want, I can:

- Jelaskan langkah lanjut tuning menggunakan grid search secara terperinci
- Berikan contoh realistis grid search untuk cardiac localization
- Jelaskan cara menghindari leakage data selama tuning dan evaluasi

ChatGPT can make mistakes. Check important info.